
✦ UD1 - UD2

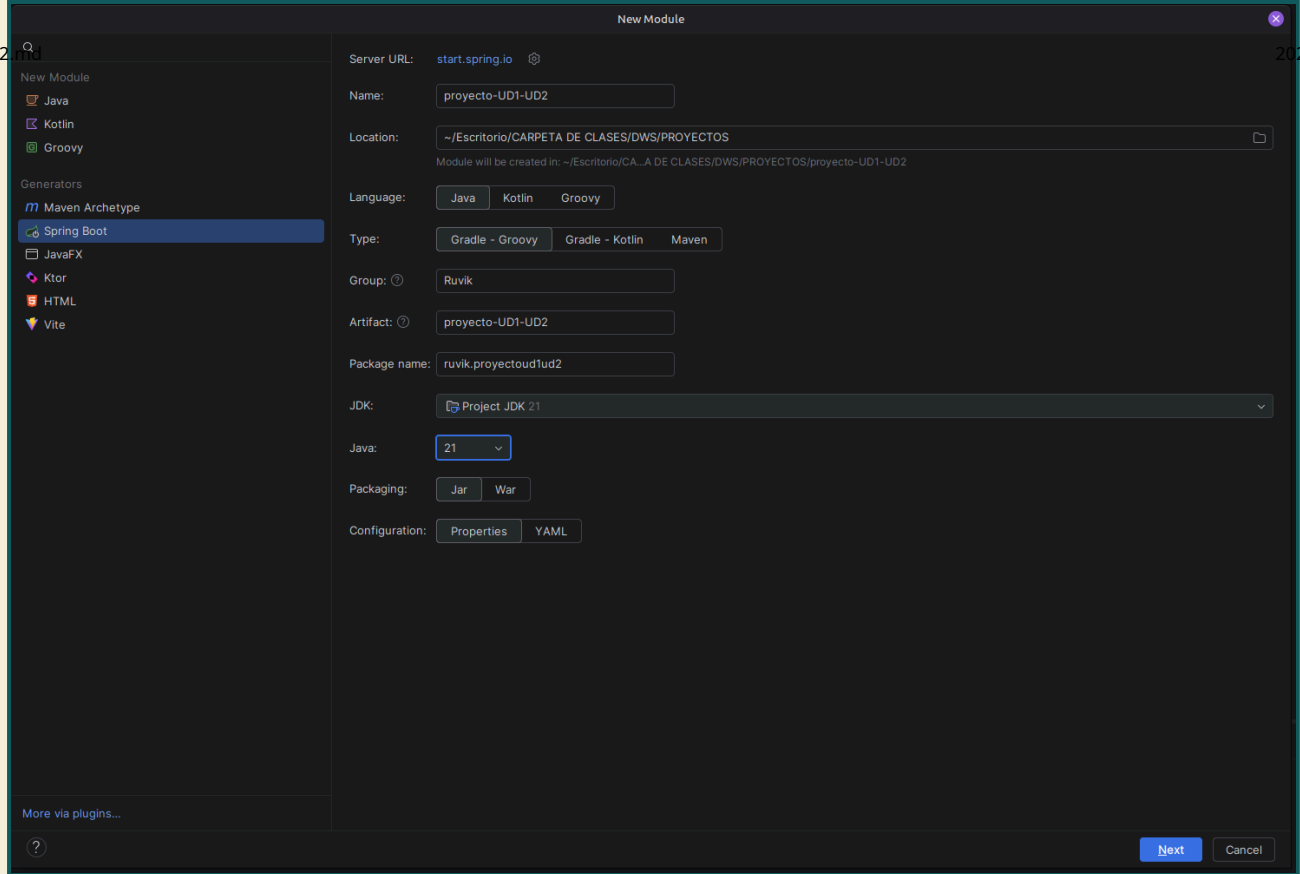
// KERIN AGUILERA

INDICE

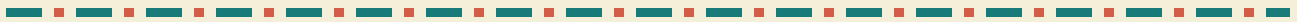
- UD1 - UD2
 - KERIN AGUILERA
 - INDICE
 - PRIMERO INTENTO EL FALLIDO
 - SEGUNDO INTENTO, EL BUENO
 - PAGINA ESTATICA
 - Spring Initializr
 - Conclusión
 - FINAL
-

PRIMERO INTENTO EL FALLIDO

Siguiente el hilo de la practica, pide que intentemos crear un proyecto **springboot** de manera vanilla, sin modificaciones e sin incluir nada.



Aqui solo haremos un cambio que es elegir Java 21.



Luego, le damos next y pasamos a la parte de la dependencias, igual no incluimos nada en ellas.

GraalVM Native Support

Support for compiling Spring applications to native executables using the GraalVM native-image compiler.

Added dependencies:

No dependencies added

Previous

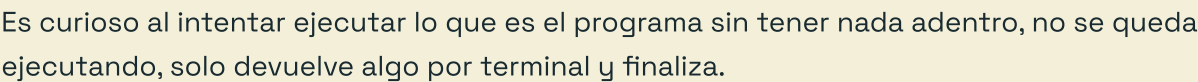
Create

Cancel

Ahora la cuestión es ¿Funciona esto? Pues de momentos nos da todo lo que puede tener una apps springboot.

tenemos la clase inicial

Con su estructura.



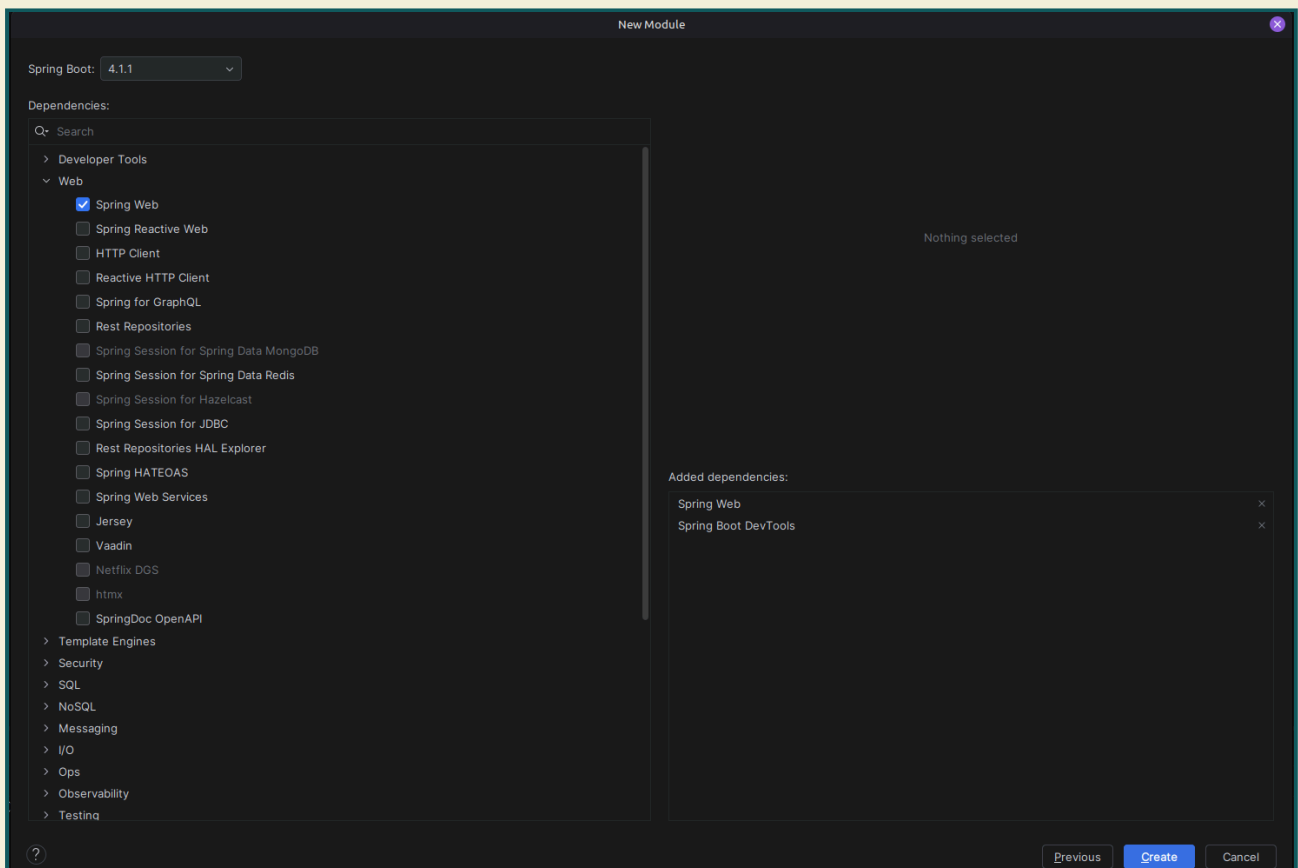
4 / 18

```
ProjectoUd1Ud2Application.java x
1 package ruvik.projectcloud1ud2;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 @RestController
8 public class ProyectoUd1Ud2Application {
9
10     public static void main(String[] args) {
11         SpringApplication.run(ProyectoUd1Ud2Application.class, args);
12     }
13
14     @GetMapping("/hello") no usages
15     public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
16         return String.format("Hello %s!", name);
17     }
18
19 }
20
```

CONCLUSIÓN TODO ESTO FALLA

SEGUNDO INTENTO, EL BUENO

Ahora volvemos a inicializar un proyecto como antes, pero ahora la peculiaridad es añadir una dependencia que lo cambia todo. **Spring web**



ahora con esto ya podemos comprar lo que sucedido en la anterior y sacar conclusiones. Y es que ahora si permite poner las anotaciones y los imports, quitando los imports sugiere lo mismo, porque aun no tenemos dependencias como JPA o mas que si tiene 2 imports distintos y

:: Spring Boot ::

```
2026-09-29T09:05:24.512+02:00 INFO 21074 --- [UD1-UD2] [ restartedMain] o.apache.catal:
```

localhost:8080 ya nos responde algo

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Tue Sep 29 09:07:11 CEST 2026

There was an unexpected error (type=Not Found, status=404).

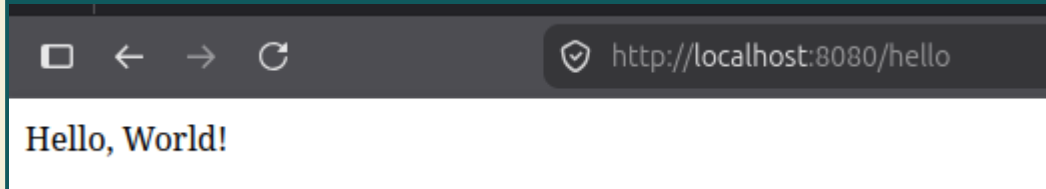
No static resource .

```
org.springframework.web.servlet.resource.NoResourceFoundException: No static resource for request '/.
    at org.springframework.web.servlet.resource.ResourceHttpRequestHandler.handleRequest(ResourceHttpRequestHandler.java:527)
    at org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(HttpRequestHandlerAdapter.java:50)
    at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:963)
    at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:866)
    at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1000)
    at org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:892)
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:622)
    at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:874)
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:132)
    at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:59)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:199)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:165)
    at org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:77)
    at org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:535)
    at org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:115)
    at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:86)
    at org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:71)
    at org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:347)
    at org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:406)
    at org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:71)
    at org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:1307)
    at org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:2036)
    at org.apache.tomcat.util.net.SocketProcessorBase.run(SocketProcessorBase.java:74)
    at org.apache.tomcat.util.threads.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:949)
    at org.apache.tomcat.util.threads.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:483)
    at org.apache.tomcat.util.threads.TaskThread$WrappingRunnable.run(TaskThread.java:74)
    at java.base/java.lang.Thread.run(Thread.java:1583)
```

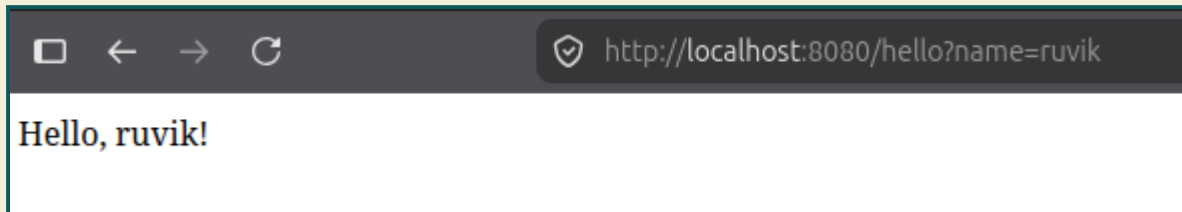
¿Pero que ocurre? Pues el fallo es claramente lógico y normal, en nuestro proyecto tenemos la siguiente línea:

```
@GetMapping("/hello")
public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
    return String.format("Hello, %s!", name);
}
```

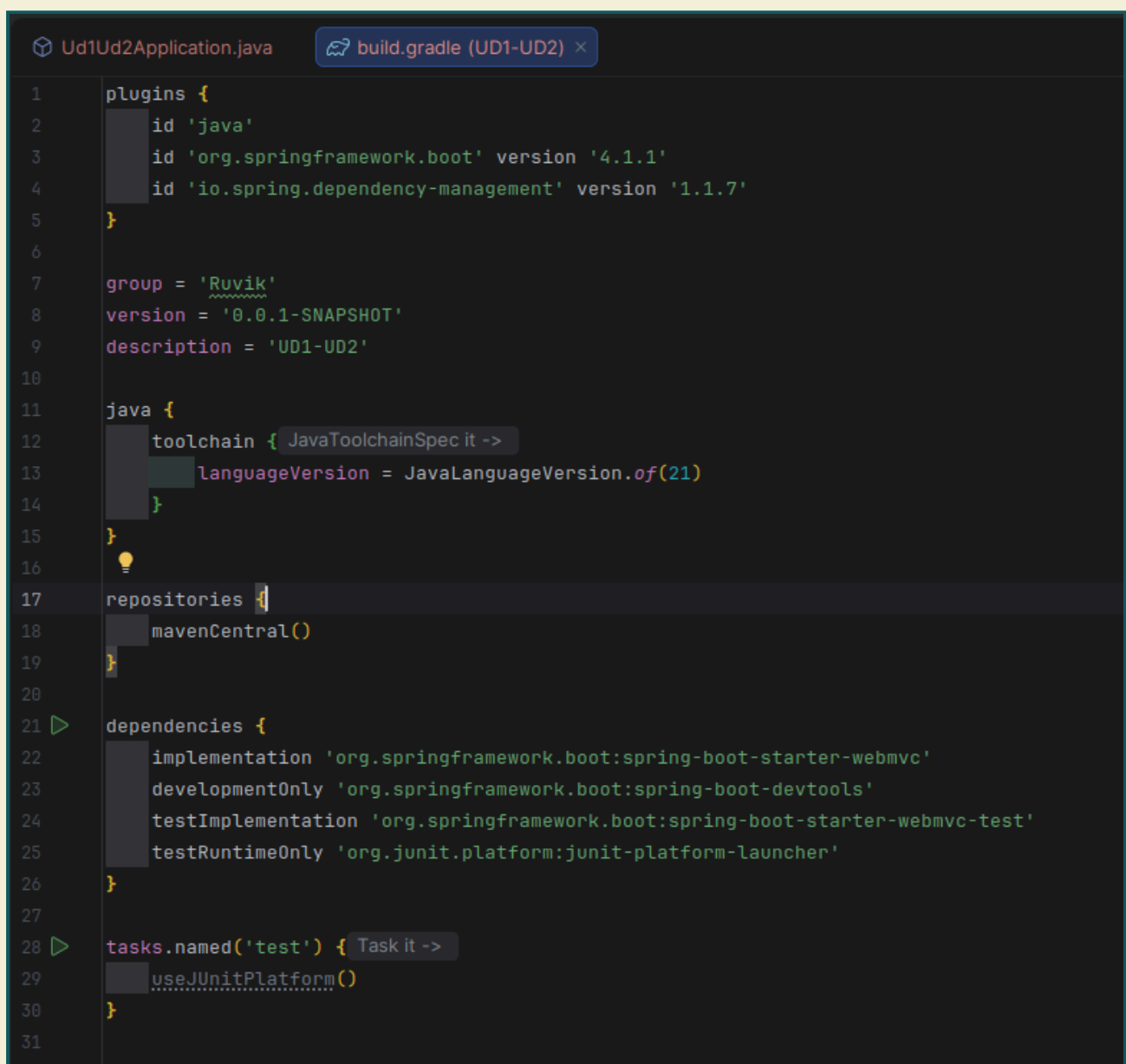
que nos dice que para visualizar *Hello World* debemos ir a la ruta `/hello` así que en el buscador deberías de poner `localhost:8080/hello` y ahora si esto debía devolvernos lo siguiente:



Y es que ahora con la pequeña logica que hay dentro, si nosotros en la url introducimos un valor en la variable name, debe saludarnos `http://localhost:8080/hello?name=ruvik`



Ya tengo la depedencias DevTools pero viendo dentro de gradle tenemos lo siguiente



en ella tenemos lo siguiente

```
plugins {  
    id 'java'  
    id 'org.springframework.boot' version '4.1.1'  
    id 'io.spring.dependency-management' version '1.1.7'  
}
```

Esto es donde determinamos la versión del framework así como el propio gradle que gestionas sus paquetes.

```
group = 'Ruvik'  
version = '0.0.1-SNAPSHOT'  
description = 'UD1-UD2'  
  
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(21)  
    }  
}  
  
repositories {  
    mavenCentral()  
}
```

aquí está todo lo que dejamos por default, viendo también que está la versión 21 de Java que elegimos

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'  
    developmentOnly 'org.springframework.boot:spring-boot-devtools'  
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'  
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'  
}
```

aquí es donde nos moveremos más, las dependencias del proyecto, donde a manos podemos añadir nuevas dependencias, o buscar dependencias sabiendo si funcionan o no.

AVISO DE RUTA //

ANOTACIÓN: todo lo que he dicho antes es segundo entiendo lo que veo. Puede existir fallos.

Al modificar tanto nombre pasar al español a inglés y sigue funcionando

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class Ud1Ud2Application {

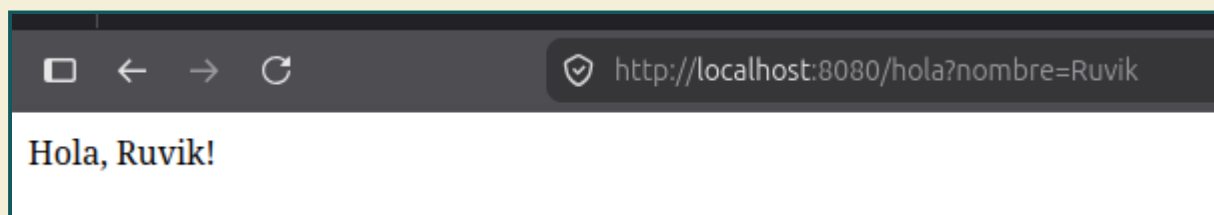
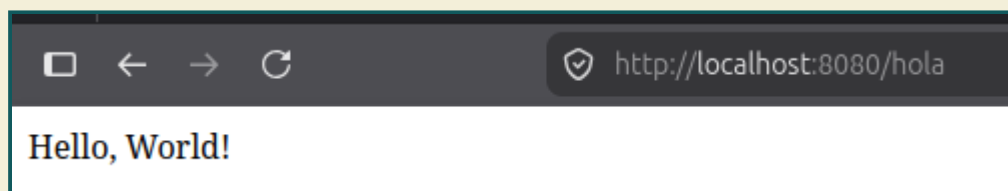
    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, %s!", nombre);
    }

}
```

Vemos el cambio de ingles a español

Y aqui abajo las repuesta de la petención `localhost:8080/hola` y `localhost:8080/hola?nombre=ruvik`



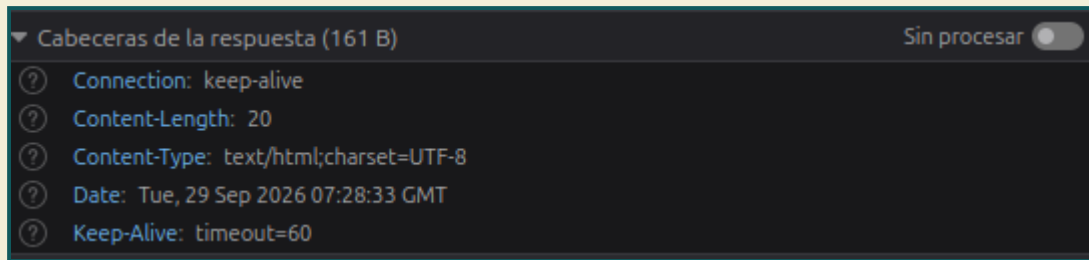
Si funciona si pones atonaciones HTML dentro de la repuesta

```
return String.format("Hola, <b> %s!</b>", nombre);
```

Si refrescamos la pagina web asi es como lo devuelve

Hola, **Ruvik!**

Y si vemos la respuesta de la cabecera esto es lo que tenemos



PAGINA ESTATICA

he puesto una pagina web estatica llamada `index.html` como aprendimos ayer esto muestra la `/` directamente este es nuestro html

```
<html>
  <body>
    <form action="/mostrar" method="post">
      <label for="nombre">Nombre:</label>
      <input type="text" name="nombre">
      <button type="submit">Enviar</button>
    </form>
  </body>
```

y este es el `@PostMapping` donde procesamos y mostramos el el mensajes:

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

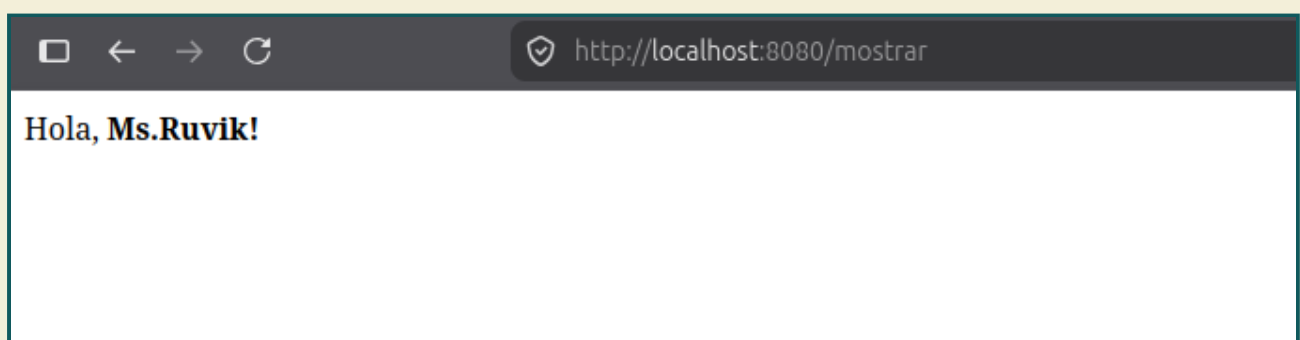
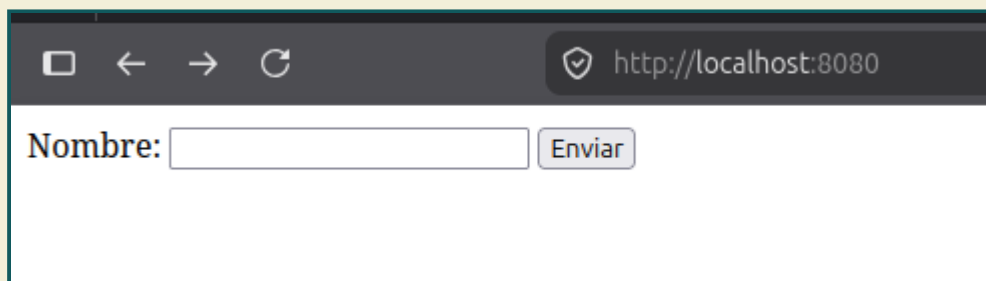
@SpringBootApplication
@RestController
public class Ud1Ud2Application {

    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, <b> %s!</b>", nombre);
    }

    @PostMapping("/mostrar")
    public String mostrar(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, <b> %s!</b>", nombre);
    }
}
```

Y aquí las capturas de cómo se muestra un formulario en  y el resultado



Primero si quitamos el Rest de RestController y lo dejamos como un Controller a secas. Lo que pasas es que a parte de que la importación cambia, aunque te siga sirviendo el html en la página principal, cuando quieres procesar algún tipo de dato falla. Por el hecho de que solo es un controller.

Pero no me acuerdo como era el año pasado, pero si he averiguado por qué me sonaba entre `@RequestBody` y `@ResponseBody` pregunte en internet sus diferencias y el

mostrar termina funcionando.

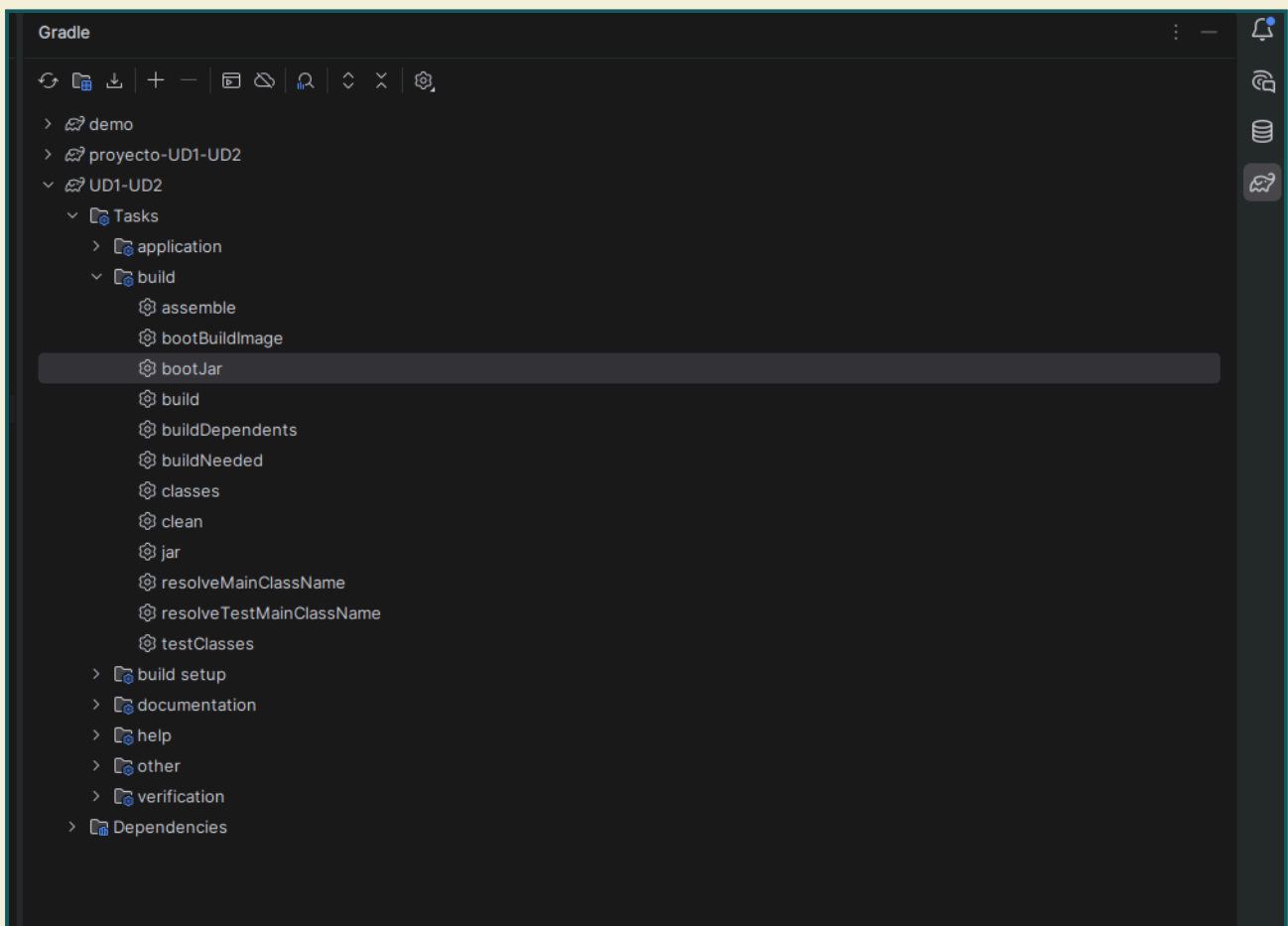
```
@PostMapping("/mostrar")
@ResponseBody
public String mostrar(@RequestParam(value = "nombre", defaultValue = "Mundo") String
    return String.format("Hola, <b> %s!</b>",name);
}
```

Spring Initializr

Lo que aporta es quitarle complejidad al crear una aplicación web que recibe peticiones, porque al utilizar el Spring Core, debemos tener muchas dependencias y ver si clases esta cableadas lo mejor posible, sino nada funciona.

Spring Initializr soluciona eso, dando una interfaz mas amable, que nos deja elegir todo y crear todo apps sin nosotros tener que preocuparnos que funciona o no, solo programar.

En IntelliJ para poder exportar el .jar es de los siguiente forma



```
vas al proyecto
  UD1-UD2
    Tasks
      build
      BootJar
```

le damos doble click y ejecuta, en la carpeta e tu proyecto esta en build y lib

para ejecutarlo debemos ir ahí por terminal

y ejecutar

```
java -jar [NOMBRE DE LA CARPETA]
```

resultado

```
alumno@daw2-07: ~/Escritorio/CARPETA DE CLASES/DWS/PROYECTOS/UD1-UD2/build/libs
alumno@daw2-07: ~/Escritorio/CARPETA DE CLASES/DWS/PROYECTOS/UD1-UD2/build/libs$ java -jar UD1-UD2-0.0.1-SNAPSHOT.jar

:: Spring Boot ::
(v4.1.1)

2026-09-29T10:04:11.143+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:11.159+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.238+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.254+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.254+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.382+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.443+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.754+02:00 INFO 33196 --- [UD1-UD2] [
2026-09-29T10:04:12.768+02:00 INFO 33196 --- [UD1-UD2] [

main| ruvik.ud1ud2.Ud1Ud2Application : Starting Ud1Ud2Application v0.0.1-SNAPSHOT using Java 21.0.11 with PID 33196 (/home/alumno/Escritorio/CARPETA DE CL
ASES/DWS/PROYECTOS/UD1-UD2/build/libs/UD1-UD2-0.0.1-SNAPSHOT.jar started by alumno in /home/alumno/Escritorio/CARPETA DE CLASES/DWS/PROYECTOS/UD1-UD2/build/libs)
main| ruvik.ud1ud2.Ud1Ud2Application : No active profile set, falling back to 1 default profile: "default"
main| o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8888 (http)
main| o.apache.catalina.core.StandardService : Starting Service [Tomcat]
main| o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.24]
main| b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1069 ms
main| o.s.b.w.a.WelcomePageHandlerMapping : Adding welcome page: class path resource [static/index.html]
main| o.s.boot.tomcat.TomcatWebServer : Tomcat started on port 8888 (http) with context path '/'
main| ruvik.ud1ud2.Ud1Ud2Application : Started Ud1Ud2Application in 2.206 seconds (process running for 2.918)
```

He buscado por internet y dice que el puerto se limita en `application.properties` he intento incluir el puerto:80

```
1 spring.application.name=UD1-UD2
2 server.port=80
```

pero parece ser que todo salta por los aires.

```
Run UD1-UD2 [ruvik.ud1ud2.Ud1Ud2Application.main()] x
[ruvik.ud1ud2.Ud1Ud2Application.main()]: successful At 29/9/26, 10:08 with 4 errors 2 sec, 981 ms
[ruvik.ud1ud2.Ud1Ud2Application.main()]: 4 errors 2 sec, 482 ms
[org.springframework.boot.web.server.WebServerException: Unable to start embedded Tomcat server]
[java.lang.IllegalArgumentException: standardService.connector.startFailed]
[org.apache.catalina.LifecycleException: Protocol handler start failed]
[java.net.BindException: Permission denied]

Caused by: org.springframework.boot.web.server.WebServerException: Unable to start embedded Tomcat server
```

Vale con el POJO introducido, devuelvo directamente el Objeto, porque habia pensado en devolver un tipo String, pero al devolver el Objeto, lo que hace Springboot es lo siguiente

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.*;
import ruvik.ud1ud2.Entity.Contacto;

@SpringBootApplication
@RestController
public class Ud1Ud2Application {

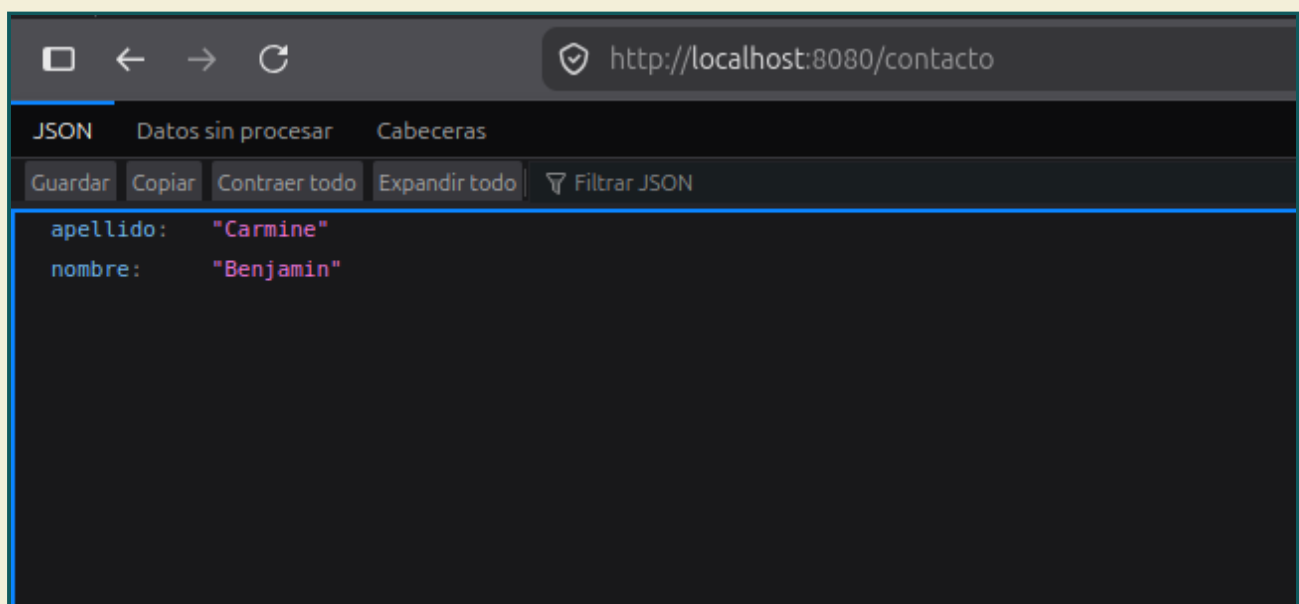
    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, <b> %s!</b>", nombre);
    }

    @PostMapping("/mostrar")
    public String mostrar(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, <b> %s!</b>", nombre);
    }

    @GetMapping("/contacto")
    public Contacto contacto(@RequestParam(value = "nombre", defaultValue = "Benjamin") String nombre,
                            @RequestParam(value = "apellido", defaultValue = "Carmine") String apellido) {
        Contacto contacto = new Contacto();
        contacto.setNombre(nombre);
        contacto.setApellido(apellido);
        return contacto;
    }
}
```

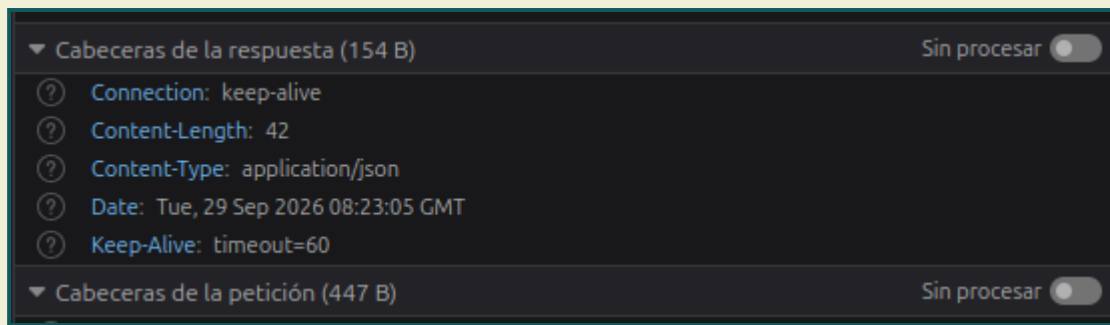
Y lo que interpreta Springboot es devolver un JSON



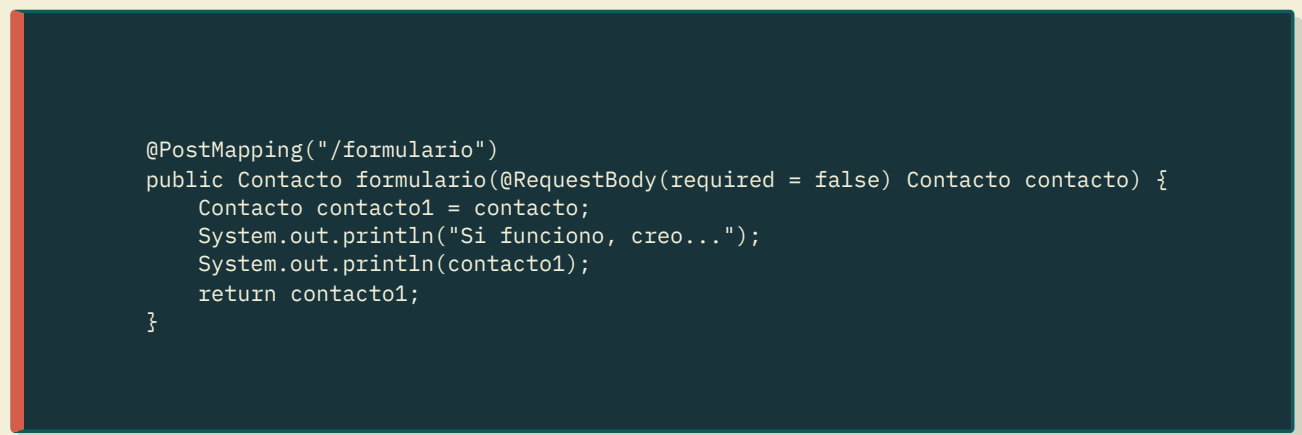
Y el Type al inspeccionar es el siguiente:

UD1-UD2.md

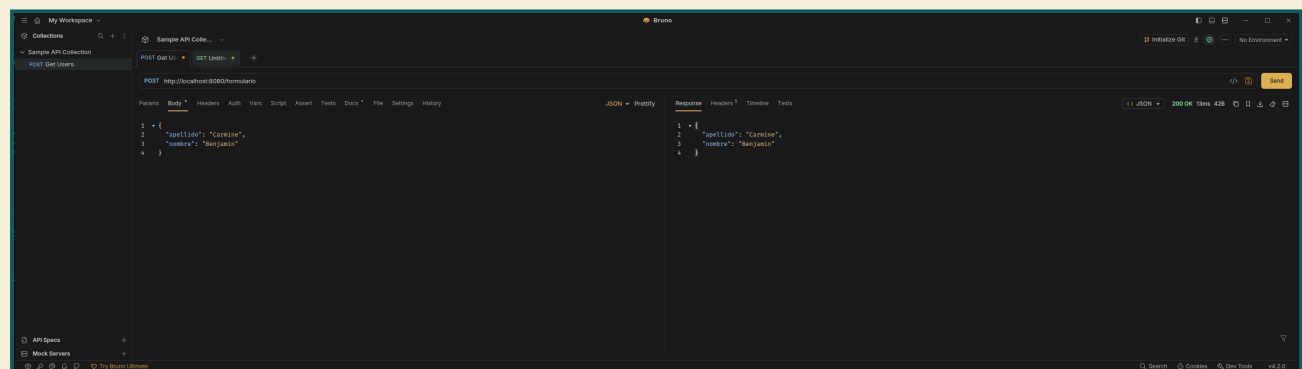
2026-10-01



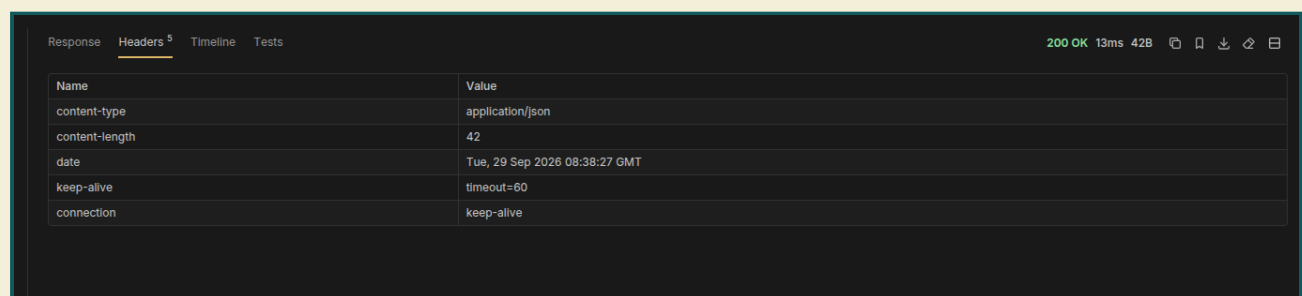
para utilizar bruno e ingresar datos por dentro del cuerpo, yo hice el `@PostMapping` lo siguiente:



Y en bruno hace lo siguiente:



Y la cabecera que devuelve es la siguiente:



Buscando información por internet, dice que se Gesntiona cuando utilizamos

16 / 18

`@RestController` o `@ResponseBody` o cuando estemos mucho mas avanzados y

`ResponseEntity` podemos de alguna manera configurar y gestionar el `typeContent`.

// Conclusión

realmente el hecho de volver a reever esto, me hizo ir mas rapido, ya entendia el porque de algunas cosas, como funciona, es verdad que me senti oxidado mas con el tema de POJOS pero en general me he sentido mas suelto y capaz de hacerlo mejor, cuando lo intente por primera vez.



FINAL
